What is Feature Engineering?

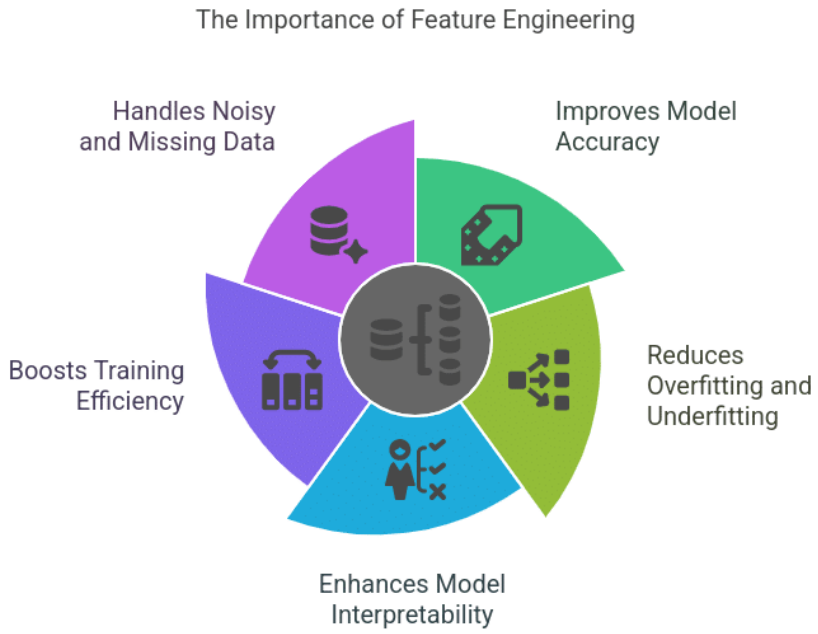
What is Feature Engineering?

Feature engineering is the art of converting raw data into useful input variables (features) that improve the performance of machine learning models. It helps in choosing the most useful features to enhance a model's capacity to learn patterns & make good predictions.

Feature engineering encompasses methods like feature scaling, encoding categorical variables, feature selection, and building interaction terms.

Why is Feature Engineering Important?

Feature engineering is one of the most critical steps in machine learning. Even the most advanced algorithms can fail if they are trained on poorly designed features. Here’s why it matters:



1. Improves Model Accuracy

A well-engineered feature set allows a model to capture patterns more effectively, leading to higher accuracy. For example, converting a date column into “day of the week” or “holiday vs. non-holiday” can improve sales forecasting models.

2. Reduces Overfitting and Underfitting

By removing irrelevant or highly correlated features, feature engineering prevents the model from memorizing noise (overfitting) and ensures it generalizes well on unseen data.

3. Enhances Model Interpretability

Features that align with domain knowledge make the model's decisions more explainable. For instance, in fraud detection, a feature like "number of transactions per hour" is more informative than raw timestamps.

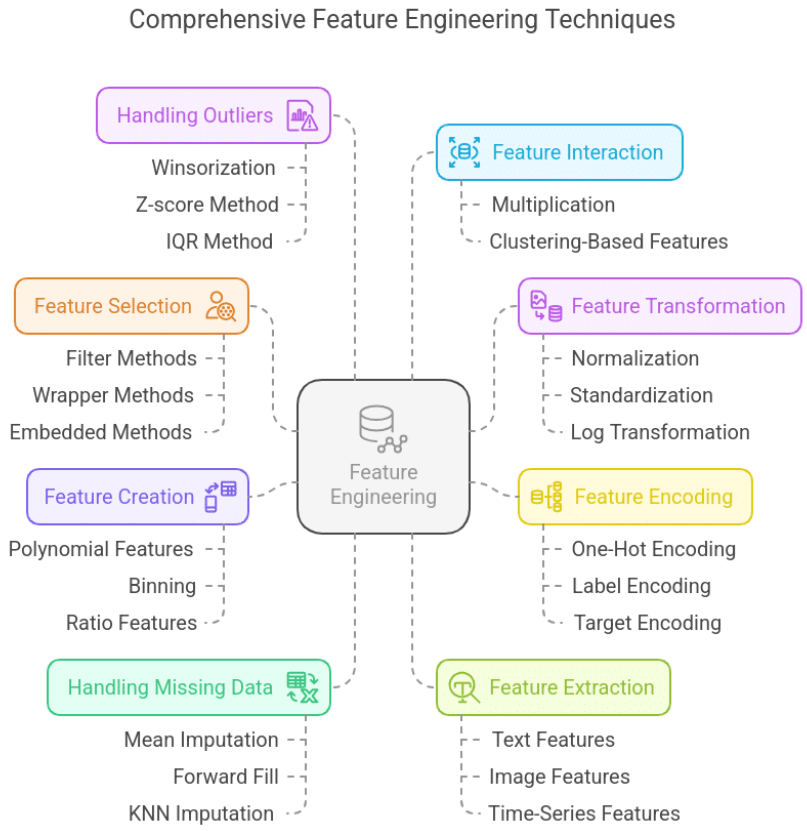
4. Boosts Training Efficiency

Reducing the number of unnecessary features decreases computational complexity, making training faster and more efficient.

5. Handles Noisy and Missing Data

Raw data is often incomplete or contains outliers. Feature engineering helps clean and structure this data, ensuring better learning outcomes.

Key Methods of Feature Engineering



1. Feature Selection

Selecting the most relevant features while eliminating redundant, irrelevant, or highly correlated variables helps improve model efficiency and accuracy.

Techniques:

- **Filter Methods:** Uses statistical techniques like correlation, variance threshold, or mutual information to select important features.
- **Wrapper Methods:** Uses iterative techniques like Recursive Feature Elimination (RFE) and stepwise selection.
- **Embedded Methods:** Feature selection is built into the algorithm, such as Lasso Regression (L1 regularization) or decision tree-based models.

Example: Removing highly correlated features like "Total Sales" and "Average Monthly Sales" if one can be derived from the other.

2. Feature Transformation

Transforms raw data to improve model learning by making it more interpretable or reducing skewness.

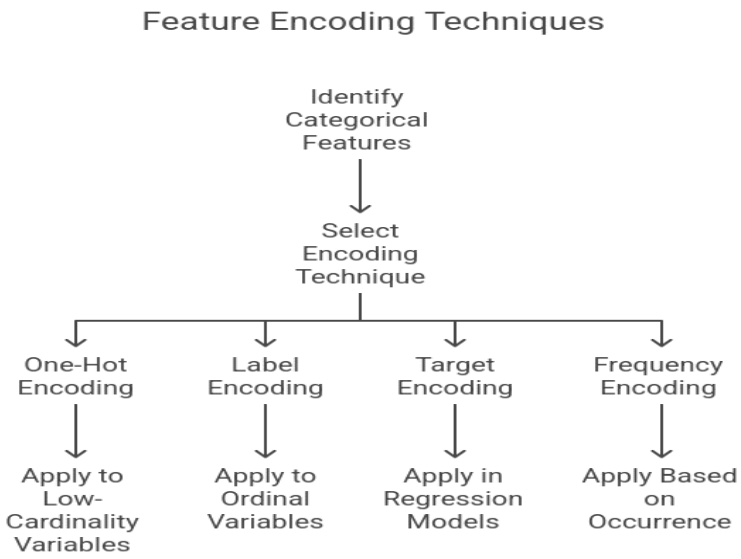
Techniques:

- **Normalization (Min-Max Scaling):** Rescales values between 0 and 1. Useful for distance-based models like k-NN.
- **Standardization (Z-score Scaling):** Transforms data to have a mean of 0 and standard deviation of 1. Works well for gradient-based models like logistic regression.
- **Log Transformation:** Converts skewed data into a normal distribution.
- **Power Transformation (Box-Cox, Yeo-Johnson):** Used to stabilize variance and make data more normal-like.

Example: Scaling customer income before using it in a model to prevent high-value dominance.

3. Feature Encoding

Categorical features must be converted into numerical values for machine learning models to process.



Techniques:

- **One-Hot Encoding (OHE):** Creates binary columns for each category (suitable for low-cardinality categorical variables).
- **Label Encoding:** Assigns numerical values to categories (useful for ordinal categories like "low," "medium," "high").
- **Target Encoding:** Replaces categories with the mean target value (commonly used in regression models).
- **Frequency Encoding:** Converts categories into their occurrence frequency in the dataset.

Example: Converting "City" into multiple binary columns using one-hot encoding:

City	New York	San Francisco	Chicago
NY	1	0	0
SF	0	1	0

4. Feature Creation (Derived Features)

Feature creation involves constructing new features from existing ones to provide additional insights and improve model performance. Well-crafted features can capture hidden relationships in data, making patterns more evident to machine learning models.

Techniques:

- **Polynomial Features:** Useful for models that need to capture non-linear relationships between variables.
  - **Example:** If a model struggles with a linear relationship, adding polynomial terms like x^2, x^3, or interaction terms (x1 * x2) can improve performance.
  - **Use Case:** Predicting house prices based on features like square footage and number of rooms. Instead of just using square footage, a model could benefit from an interaction term like square_footage * number_of_rooms.

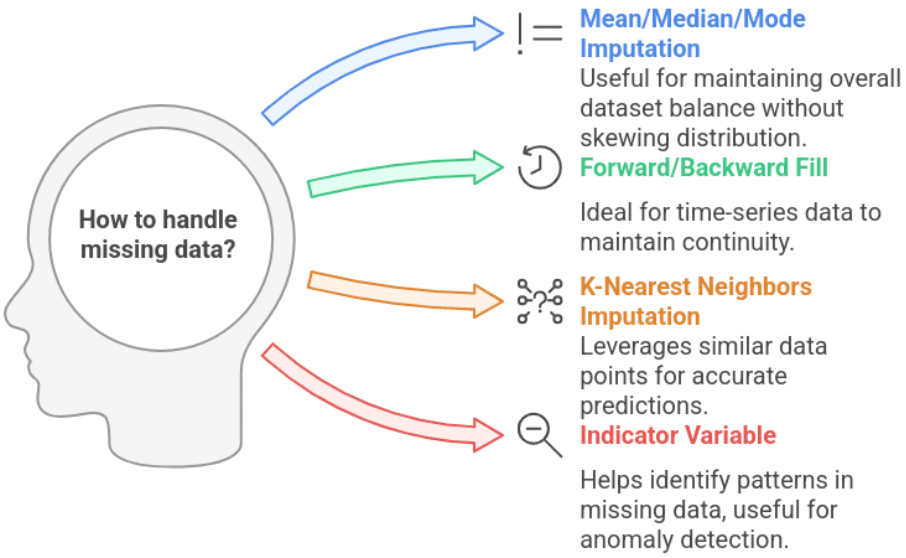
- **Binning (Discretization):** Converts continuous variables into categorical bins to simplify the relationship.
  - **Example:** Instead of using raw age values (22, 34, 45), we can group them into bins:
    - Young (18-30)
    - Middle-aged (31-50)
    - Senior (51+)
  - **Use Case:** Credit risk modeling, where different age groups have different risk levels.
- **Ratio Features:** Creating ratios between two related numerical values to normalize the impact of scale.
  - Example: Instead of using income and loan amount separately, use Income-to-Loan Ratio = Income / Loan Amount to standardize comparisons across different income levels.
  - **Use Case:** Loan default prediction, where individuals with a higher debt-to-income ratio are more likely to default.
- **Time-based Features:** Extracts meaningful insights from timestamps, such as:
  - Hour of the day (helps in traffic analysis)
  - Day of the week (useful for sales forecasting)
  - Season (important for retail and tourism industries)
  - **Use Case:** Predicting e-commerce sales by analyzing trends based on weekdays vs. weekends.

Example:

Timestamp	**Hour**	**Day of Week**	**Month**	**Season**
2024-02-15 14:30	14	Thursday	2	Winter
2024-06-10 08:15	8	Monday	6	Summer

5. Handling Missing Data

Missing data is common in real-world datasets and can negatively impact model performance if not handled properly. Instead of simply dropping missing values, feature engineering techniques help retain valuable information while minimizing bias.



Techniques:

- **Mean/Median/Mode Imputation:**
  - Fills missing values with the mean (for numerical data) or mode (for categorical data).
  - **Example:** Filling missing salary values with the median salary of the dataset to prevent skewing the distribution.
- **Forward or Backward Fill (Time-Series Data):**
  - **Forward fill:** Uses the last known value to fill missing entries.
  - **Backward fill:** Uses the next known value to fill missing entries.
  - **Use Case:** Stock market data where missing prices can be filled with the previous day's prices.
- **K-Nearest Neighbors (KNN) Imputation:**

- Uses similar data points to estimate missing values.
- **Example:** If a person's income is missing, KNN can predict it based on people with similar job roles, education levels, and locations.

- **Indicator Variable for Missingness:**
  - Creates a binary column (1 = Missing, 0 = Present) to retain missing data patterns.
  - **Use Case:** Detecting fraudulent transactions where missing values themselves may indicate suspicious activity.

Example:

Customer ID	**Age**	**Salary**	**Salary Missing Indicator**
101	35	50,000	0
102	42	NaN	1
103	29	40,000	0

6. Feature Extraction

Feature extraction involves deriving new, meaningful representations from complex data formats like text, images, and time-series. This is especially useful in high-dimensional datasets.

Techniques:

- **Text Features:** Converts textual data into numerical form for machine learning models.
  - **Bag of Words (BoW):** Represents text as word frequencies in a matrix.
  - **TF-IDF (Term Frequency-Inverse Document Frequency):** Gives importance to words based on their frequency in a document vs. overall dataset.

    - **Word Embeddings (Word2Vec, GloVe, BERT):** Captures semantic meaning of words.
    - **Use Case:** Sentiment analysis of customer reviews.
- **Image Features:** Extract essential patterns from images.
    - **Edge Detection:** Identifies object boundaries in images (useful in medical imaging).
    - **Histogram of Oriented Gradients (HOG):** Used in object detection.
    - **CNN-based Feature Extraction:** Uses deep learning models like ResNet and VGG for automatic feature learning.
    - **Use Case:** Facial recognition, self-driving car object detection.
- **Time-Series Features:** Extract meaningful trends and seasonality from time-series data.
    - **Rolling Averages:** Smooth out short-term fluctuations.
    - **Seasonal Decomposition:** Separates trend, seasonality, and residual components.
    - **Autoregressive Features:** Uses past values as inputs for predictive models.
    - **Use Case:** Forecasting electricity demand based on historical consumption patterns.
- **Dimensionality Reduction (PCA, t-SNE, UMAP):**
    - **PCA (Principal Component Analysis)** reduces high-dimensional data while preserving variance.
    - **t-SNE and UMAP** are useful for visualizing clusters in large datasets.
    - **Use Case:** Reducing thousands of customer behavior variables into a few principal components for clustering.

Example:

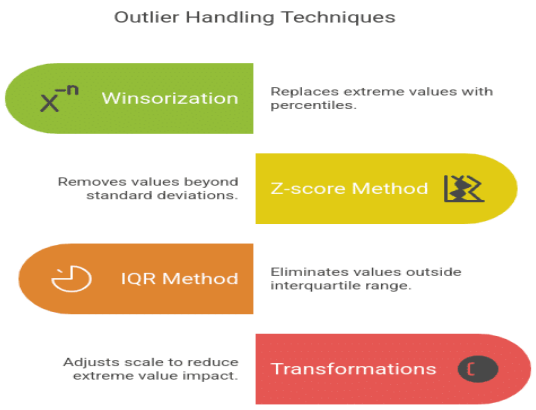
For text analysis, TF-IDF converts raw sentences into numerical form:

Sentence	**"AI is transforming healthcare"**	**"AI is advancing research"**

AI	0.4	0.3
transforming	0.6	0.0
research	0.0	0.7

7. Handling Outliers

Outliers are extreme values that can distort a model's predictions. Identifying and handling them properly prevents skewed results.



Techniques:

- **Winsorization:** Replaces extreme values with a specified percentile (e.g., capping values at the 5th and 95th percentile).
- **Z-score Method:** Removes values that are more than a certain number of standard deviations from the mean (e.g., ±3σ).
- **IQR (Interquartile Range) Method:** Removes values beyond 1.5 times the interquartile range (Q1 and Q3).
- **Transformations (Log, Square Root):** Reduces the impact of extreme values by adjusting scale.

Example:

Detecting outliers in a salary dataset using IQR:

Employee	Salary	Outlier (IQR Method)
A	50,000	No
B	52,000	No
C	200,000	Yes

8. Feature Interaction

Feature interaction helps capture relationships between variables that aren't obvious in their raw form.

Techniques:

- **Multiplication or Division of Features:**
  - Example: Instead of using "Weight" and "Height" separately, create BMI = Weight / Height².
- **Polynomial Features:**
  - Example: Adding squared or cubic terms for better non-linear modelling.
- **Clustering-Based Features:**
  - Assign cluster labels using k-means, which can be used as categorical inputs.

Example:

Creating an "Engagement Score" for a user based on:

1 Engagement Score = (Logins * Time Spent) / (1 + Bounce Rate).

By leveraging these feature engineering techniques, you can transform raw data into powerful predictive inputs, ultimately improving model accuracy, efficiency, and interoperability.

Feature Engineering Pipelines:

Feature engineering pipelines are **not the same** for all types of data. Each data type requires **different preprocessing steps**, **different transformations**, and **different feature extraction methods**.

(a) Numerical Data

Numerical data includes values such as age, salary, distance, marks, temperature, etc.

How the Pipeline Works

1. **Handle missing values**
    - Using mean, median, interpolation
2. **Remove or treat outliers**
    - Winsorizing, clipping
3. **Scaling or normalization**
    - Bringing values into a standard range
4. **Transformation**
    - Log, square root, power transform
5. **Feature construction**
    - Creating polynomial, interaction features

Two Important Techniques

1. Min–Max Scaling

Brings values to range **0–1**

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

2. Standardization (Z-Score)

Centers values to **mean = 0**, **std = 1**

$$z = \frac{x - \mu}{\sigma}$$

Other Numerical Techniques

- Binning (equal width, equal frequency)
- Log transformation
- Polynomial features

(b) Categorical Data

Categorical data includes values like gender, city, color, department, grade, etc.

How the Pipeline Works

1. **Handle missing categories**
    - Using mode imputation or “Unknown” label
2. **Detect rare categories**
    - Combine as “Other”
3. **Encode categorical values**
4. **Avoid data leakage or overfitting from categories**
5. **Check for sensitive attributes**
    - Gender, religion → handle ethically

Two Important Techniques

1. One-Hot Encoding

Creates a new binary column for each category.
Best for: small number of categories.

Example:
"Red", "Blue" → Red=1/0, Blue=1/0

2. Label Encoding

Converts categories into numbers (0,1,2...).
Best for: ordinal data (e.g., Low < Medium < High)

Other Categorical Techniques

- Target encoding
- Frequency encoding
- Binary encoding

- Embedding-based encoding (for NLP)

(c) Time-Series Data

Time-series data includes values collected over time: stock prices, sensor readings, weather data, sales data.

How the Pipeline Works

1. **Convert date/time into useful components**
    - Year, month, hour, day-of-week
2. **Handle missing timestamps**
    - Forward fill, backward fill, interpolation
3. **Detect & treat trends and seasonality**
4. **Create lag-based and window-based features**
5. **Ensure data is not shuffled**
    - Split using time-based train-test split

Two Important Techniques

1. Lag Features

Using past observations as new features.
Example:
Lag 1 = previous day's value
Lag 7 = previous week's value

Makes models learn patterns over time.

2. Rolling Window Statistics

Calculate moving averages, moving sums, moving variance, etc.

Examples:

- 7-day moving average
- 30-day moving standard deviation

Other Time-Series Techniques

- Seasonal decomposition
- Fourier features
- Time difference features
- Exponential smoothing features

Data Type	Pipeline Focus	Two Main Techniques
Numerical	Scaling, outlier treatment, transformation	Min–Max Scaling, Standardization
Categorical	Encoding categories, handling rare values	One-Hot Encoding, Label Encoding
Time-Series	Time-based splits, lag/rolling features	Lag Features, Rolling Window Stats

Comparison: Good vs. Bad Feature Engineering

Data Preprocessing

Good Feature Engineering

Handles missing values, removes outliers, and applies proper scaling.

Bad Feature Engineering

Ignores missing values, includes outliers, and fails to standardize.

Feature Selection

Good Feature Engineering

Uses correlation analysis, importance scores, and domain expertise to pick features.

Bad Feature Engineering

Uses all features, even if some are redundant or irrelevant.

Feature Transformation

Good Feature Engineering

Normalizes, scales, and applies log transformations when necessary.

Bad Feature Engineering

Uses raw data without processing, leading to inconsistent model behavior.

Encoding Categorical Data

Good Feature Engineering

Uses proper encoding techniques like one-hot, target, or frequency encoding.

Bad Feature Engineering

Assigns arbitrary numeric values to categories, misleading the model.

Feature Creation

Good Feature Engineering

Introduces meaningful new features (e.g., ratios, interactions, polynomial terms).

Bad Feature Engineering

Adds random variables or duplicates existing features.

Handling Time-based Data

Good Feature Engineering

Extracts useful patterns (e.g., day of the week, trend indicators).

Bad Feature Engineering

Leaves timestamps in raw format, making patterns harder to learn.

Model Performance

Good Feature Engineering

Higher accuracy, generalizes well on new data, interpretable results.

Bad Feature Engineering

Poor accuracy, overfits training data, fails in real-world scenarios.

How Does Good and Bad Feature Engineering Affect Model Performance?

Example 1: Predicting House Prices

- **Good Feature Engineering**: Instead of using the raw "Year Built" column, create a new feature: **"House Age" (Current Year - Year Built)**. This provides a clearer relationship with price.
- **Bad Feature Engineering**: Keeping the "Year Built" column as-is, forcing the model to learn complex patterns instead of focusing on a straightforward numerical relationship.

Example 2: Credit Card Fraud Detection

- **Good Feature Engineering**: Creating a new feature **"Number of Transactions in the Last Hour"** helps identify suspicious activity.
- **Bad Feature Engineering**: Using raw timestamps without any transformation, making it difficult for the model to detect time-based anomalies.

Example 3: Customer Churn Prediction

- **Good Feature Engineering**: Combining customer interaction data into a **"Monthly Activity Score"** (e.g., logins, purchases, support queries).
- **Bad Feature Engineering**: Using each interaction type separately, making the dataset unnecessarily complex and harder for the model to interpret.

1. Understanding the Business Problem Before Selecting Features

Before applying feature engineering techniques, it's crucial to understand the specific problem the model aims to solve. Selecting features without considering the domain context can lead to irrelevant or misleading inputs, reducing model effectiveness.

Best Practices:

- Collaborate with domain experts to identify key variables.
- Analyze historical trends and real-world constraints.
- Ensure selected features align with business objectives.

Example:
For a loan default prediction model, features like credit score, income stability, and past loan repayment behavior are more valuable than generic features like ZIP code.

2. Avoiding Data Leakage Through Careful Feature Selection

Data leakage occurs when information from the training set is inadvertently included in the test set, leading to overly optimistic performance that does not generalize well to real-world scenarios.

Best Practices:

- Exclude features that wouldn't be available at the time of prediction.
- Avoid using future information in training data.
- Be cautious with derived features based on target variables.

Example of Data Leakage:
Using **"total purchases in the next 3 months"** as a feature to predict customer churn. Since this information wouldn't be available at the prediction time, it would lead to incorrect model evaluation.

Fix: Use past purchase behavior (e.g., "number of purchases in the last 6 months") instead.

3. Experimenting with Different Transformations and Encodings

Transforming features into more suitable formats can significantly enhance model performance. This includes scaling numerical variables, encoding categorical variables, and applying mathematical transformations.

Best Practices:

- **Scaling numerical variables** (Min-Max Scaling, Standardization) to ensure consistency.
- **Encoding categorical variables** (One-Hot, Label Encoding, Target Encoding) based on data distribution.
- **Applying transformations** (Log, Square Root, Power Transform) to normalize skewed data.

Example:
For a dataset with highly skewed income data:

- **Raw Income Data:** [10,000, 20,000, 50,000, 1,000,000] (skewed distribution)
- **Log Transformation:** [4, 4.3, 4.7, 6] (reduces skewness, improving model performance).

4. Validating Engineered Features Using Visualization and Correlation Analysis

Before finalizing features, it's essential to validate their impact using exploratory data analysis (EDA) techniques.

Best Practices:

- Use **histograms and box plots** to check feature distributions.
- Use **scatter plots and correlation heatmaps** to identify relationships between variables.

- Remove highly correlated features to prevent multicollinearity.

Example:
In a sales prediction model, if "**marketing spend**" and "**ad budget**" have a **correlation > 0.9**, keeping both could introduce redundancy. Instead, use one or create a derived feature like "**marketing efficiency = revenue/ad spend**".

5. Testing Features with Different Models to Evaluate Impact

Feature importance varies across different algorithms. A feature that improves performance in one model might not be useful in another.

Best Practices:

- Train multiple models (Linear Regression, Decision Trees, Neural Networks) and compare feature importance.
- Use **Permutation Importance** or **SHAP values** to understand each feature's contribution.
- Perform **Ablation Studies** (removing one feature at a time) to measure performance impact.

Example:
In a customer churn model:

- Decision trees might prioritize customer complaints and contract types.
- Logistic regression might find tenure and monthly bill amounts more significant. By testing different models, you can identify the most valuable features.

Solved Examples:

Example 1: Employee Productivity Dataset

Dataset

Employee	Hours Worked	Productivity Score	Department	Experience Level
1	40	85	Sales	Junior
2	—	78	HR	Senior
3	38	—	Marketing	Junior

Missing Value Imputation

Ethical Imputation Strategy

To maintain fairness and avoid introducing bias:

• Numerical Features (Hours Worked, Productivity Score)

Use **median imputation** because:

- Median is robust to outliers
- It does not distort the data distribution
- It treats all employees fairly without assuming behavior

• Categorical Features (Department, Experience Level)

Use **mode imputation** because:

- It fills missing values based on the most common category
- Avoids making personal assumptions
- Does not violate ethical concerns regarding sensitive attributes

Example of Imputation

1. Missing* Hours Worked *(Employee 2)

Values available: 40 and 38
Median = (38 + 40) / 2 = **39**

2. Missing* Productivity Score *(Employee 3)

Values available: 85 and 78
Median = (78 + 85) / 2 = **81.5**

Updated Dataset After Imputation

Employee	Hours Worked	Productivity Score	Department	Experience
1	40	85	Sales	Junior
2	39	78	HR	Senior
3	38	81.5	Marketing	Junior

Scaling using Min–Max Scaling

We apply scaling to **Hours Worked** and **Productivity Score** to bring values to the range 0–1.

Min–Max Formula

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Step 1: Find Min and Max

Hours Worked:

- Min = 38
- Max = 40

Example Calculation for Hours Worked = 40:

$$x' = \frac{40 - 38}{40 - 38} = \frac{2}{2} = 1$$

Scaled Hours Worked

Value	Formula	Scaled Result
40	(40−38)/2	1
39	(39−38)/2	0.5
38	(38−38)/2	0

(Similar scaling can be applied to Productivity Score.)

Categorical Encoding using One-Hot Encoding

We convert categorical columns into numerical binary columns.

Department Categories:

- Sales
- HR
- Marketing

Experience Level Categories:

- Junior
- Senior

One-Hot Encoding Output Example

Employee	Dept_Sales	Dept_HR	Dept_Marketing	Exp_Junior	Exp_Senior
1	1	0	0	1	0

Employee	Dept_Sales	Dept_HR	Dept_Marketing	Exp_Junior	Exp_Senior
2	0	1	0	0	1
3	0	0	1	1	0

Ethical Considerations

- Avoid encoding sensitive attributes such as gender, religion, or caste.
- Before encoding, check if the feature may introduce bias in the model.
- If sensitive, apply fairness techniques (removal, grouping, debiasing).

Example: 2

Dataset:

Values = **[5, 12, 18, 25]**

Tasks:

1. Apply **Min–Max Scaling** and show all step-by-step calculations.
2. Apply **Standardization (Z-score Normalization)** by computing the **mean**, **standard deviation**, and **Z-scores** for each value.
3. Compare both scaling techniques and state **when Min–Max Scaling is preferred** and **when Standardization is preferred**.

Dataset:

Values = **[5, 12, 18, 25]**

---

1. Min–Max Scaling

Formula

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Step 1: Find Minimum and Maximum

- $x_{\min} = 5$
- $x_{\max} = 25$

Range = $25 - 5 = 20$

Step 2: Apply Min–Max Scaling

Value (x)	Formula	Scaled Value (x′)
5	(5 − 5) / 20	0
12	(12 − 5) / 20	7/20 = **0.35**
18	(18 − 5) / 20	13/20 = **0.65**
25	(25 − 5) / 20	20/20 = **1**

Scaled Values:

[0, 0.35, 0.65, 1]

2. Standardization (Z-Score Normalization)

Formula

$$z = \frac{x - \mu}{\sigma}$$

Step 1: Compute Mean

$$\mu = \frac{5 + 12 + 18 + 25}{4} = \frac{60}{4} = 15$$

Step 2: Compute Standard Deviation

Variance formula:

$$\sigma^2 = \frac{(x-\mu)^2}{N}$$

$$\sigma^2 = \frac{(5-15)^2 + (12-15)^2 + (18-15)^2 + (25-15)^2}{4}$$

$$= \frac{100 + 9 + 9 + 100}{4}$$

$$= \frac{218}{4} = 54.5$$

Standard deviation:

$$\sigma = \sqrt{54.5} \approx 7.38$$

Step 3: Compute Z-Scores

Value (x)	Formula	Z-Score
5	(5 − 15)/7.38	−10/7.38 = **−1.36**
12	(12 − 15)/7.38	−3/7.38 = **−0.41**
18	(18 − 15)/7.38	3/7.38 = **0.41**
25	(25 − 15)/7.38	10/7.38 = **1.36**

Standardized Values:

[−1.36, −0.41, 0.41, 1.36]

3. Comparison & When to Use Each

Min–Max Scaling

- Converts values into **range [0, 1]**
- Preserves original shape of distribution
- **Highly affected by outliers**

Use when:

- Algorithms require bounded range (e.g., Neural Networks, KNN)
- No extreme outliers in data

Standardization (Z-Score Normalization)

- Converts values to mean = 0 and standard deviation = 1
- Not restricted to a fixed range
- **More robust in presence of outliers**

Use when:

- Data has outliers
- Algorithms assume normal distribution or distance-based learning (e.g., Logistic Regression, SVM, PCA)

Role of Ethical Data Preprocessing in a Feature Engineering Pipeline

Ethical data preprocessing ensures that the data used for model training does **not introduce discrimination, unfair predictions, or biased outcomes**.
It protects individuals' privacy, maintains fairness, and improves trust in AI systems.

1. Handling Sensitive Attributes

Sensitive attributes include: **Gender, Age, Race, Religion, Caste, Disability Status, Income Level, Location**, etc.

Role in Ethical Feature Engineering

- Prevent unfair predictions or discrimination
- Avoid legal and ethical violations
- Ensure models do not learn harmful patterns

Examples (Multiple)

✔ Job Recruitment System

- Removing *Gender* so hiring is based only on skills.

✔ Loan Approval Dataset

- Masking *Race* and *Religion* because they should not influence credit decisions.

✔ Medical Diagnosis Dataset

- Keeping *Age* but converting it into **age ranges** (0–20, 20–40, 40–60) to protect privacy.

✔ **School Admission Dataset**

- Removing *Caste* attribute before training to avoid biased admission decisions.

✔ **Insurance Pricing Dataset**

- Protecting *Disability Status* by excluding it or anonymizing.

- Prevents models from making unfair predictions
- Reduces discrimination in decision-making systems
- Ensures legal and ethical compliance

Examples

- **Removing Gender** from a job-shortlisting dataset to avoid biased hiring.
- Replacing **Caste/Religion** with a neutral category like *"Not Used for Modeling"*.
- Masking **Age** to avoid age-based bias in loan approval.

---

2. Bias Removal Techniques

Bias can enter data due to imbalance, historical discrimination, or skewed sampling.

Bias arises due to:

- Historical injustice
- Non-representative sampling
- Human labeling errors
- Imbalanced data

Role in Ethical Feature Engineering

- Ensure equal treatment across groups
- Avoid reinforcing social inequalities
- Improve model accuracy & fairness

Examples (Multiple)

✔ **Credit Scoring**

- If historically women get fewer approvals, the model may learn this bias. → Use **reweighting** or **resampling** to balance approval samples.

✔ **Facial Recognition Dataset**

- If dataset has 90% light-skinned faces and 10% dark-skinned faces → Apply **oversampling**, **data augmentation**, or **collect balanced data**.

✔ Employee Performance Dataset

- If older workers are underrepresented, sampling bias may appear. → Add synthetic samples or normalize age representation.

✔ College Admission Model

- If certain categories have fewer examples historically → Apply **stratified sampling** to balance representation.

✔ Healthcare Data

- Chronic disease datasets often underrepresent rural populations. → Rebalance using oversampling techniques.

Examples

- **Rebalancing the dataset** (e.g., equal number of male and female applicants).
- **Removing skewed historical patterns**, such as lower salaries for certain communities.
- Using **Fair Sampling**, ensuring all demographic groups are represented.

Common Bias Removal Techniques

- Oversampling underrepresented groups
- Undersampling overrepresented groups
- Reweighting samples
- Detecting and removing discriminatory patterns

---

3. Fair Encoding Methods

Encoding can unintentionally introduce bias if not handled properly.

Encoding can unintentionally create unfairness if not done carefully.

Role in Ethical Feature Engineering

- Prevents models from learning numerical meaning where none exists
- Removes ordering or hierarchy in categorical attributes
- Supports fair decision-making

Examples (Multiple)

✔ Gender (Male, Female, Other)

- Avoid Label Encoding (Male=1, Female=2 → false hierarchy)
- Use **One-Hot Encoding** or **Binary Encoding**

✔ Location-Based Credit Risk

- Avoid treating regions with small populations unfairly
- Use **frequency encoding** so rare categories don't get extreme weights

✔ Education Level (High School < Graduate < Postgraduate)

- Use **Ordinal Encoding** because there is a true order
- But ensure that the numeric gap reflects logical order, not bias

✔ Customer Segments (Bronze, Silver, Gold)

- Use **Target Encoding** safely (with smoothing) to avoid leakage

✔ Job Titles

- High-cardinality (many unique names)
  → Use **embedding-based encoding** or **clustering job titles**

Examples

- Avoid **Label Encoding** for sensitive attributes like gender
  (Male=1, Female=0 creates hierarchy).
- Use **One-Hot Encoding** to avoid ordering bias:
  Gender → Male:1/0, Female:1/0
- Use **Target Encoding** only after checking for leakage and bias.
- For high-cardinality features (like location), use **frequency encoding** to reduce bias.

Fair Encoding Methods

- One-Hot Encoding (fair for non-ordinal data)
- Binary Encoding (reduces hierarchy)
- Removing sensitive categories before encoding
- Ethical grouping (e.g., combine rare categories to avoid unfair treatment)